



Usopp Challenge

Montar el entorno del desafío

Primero, debemos iniciar el entorno para completar el desafío para ello es necesario levantar el contenedor de docker utilizando el primer comando documentado en  [Utils](#).

Luego, debemos cambiar de usuario al correspondiente al reto, para ello utilizamos el segundo comando documentado en  [Utils](#).

```
PS C:\Users\Usuario\Documents\GitHub\data-encryption\ctf_onepice_symmetric_cipher> docker exec -it usopp_challenge bash
nobody@c1ebee8fa1f1:/home/usopp$ su usopp
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

usopp@c1ebee8fa1f1:~$
```

La contraseña es la bandera del  [Zoro Challenge](#).

Extraer bandera de **flag.txt**

El primer paso para extraer la bandera del desafío, es encontrar todos los posibles archivos que puedan contenerla, para ello, utilizamos el primer comando documentado en  [Utils](#).

```
usopp@c1ebee8fa1f1:~/ONEPIECE$ find . -type f -iname "*flag*"
./East_Blue/Syrup_Village/Casa_de_Kaya/flag.txt
./East_Blue/Baratie/Casa_de_Sanji/flag.txt
./East_Blue/Arlong_Park/Casa_de_Genzo/flag.txt
./Alabasta/Spiders_Cafe/Casa_de_Mr. 0/flag.txt
./Alabasta/Yuba/Casa_de_Kohza/flag.txt
./Alabasta/Nanohana/Casa_de_Pell/flag.txt
./Whole_Cake_Island/Caramel_Mountain/Casa_de_Katakuri/flag.txt
./Whole_Cake_Island/Cacao_Island/Casa_de_Katakuri/flag.txt
./Whole_Cake_Island/Liqueur_Island/Casa_de_Pudding/flag.txt
./Water_7/Carpenters_Cafe/Casa_de_Lucci/flag.txt
./Pirate_Island/Pirates_Ship/Casa_de_Shanks/flag.txt
usopp@c1ebee8fa1f1:~/ONEPIECE$
```

Luego de obtener todos los potenciales archivos, verificamos el contenido de cada uno, utilizando el comando `cat`, hasta encontrar el archivo que contiene texto cifrado.

```
usopp@c1ebee8fa1f1:~/ONEPIECE$ cat ./Alabasta/Nanohana/Casa_de_Pell/flag.txt
a77742694e1806841d6b3731d7ce8e2a166a5c63c11c1788424f27b941adc084bb4802d939usopp@c1ebee8fa1f1:~/ONEPIECE$
usopp@c1ebee8fa1f1:~/ONEPIECE$
```

Para descifrar la bandera, utilizamos un script en Python que implementa un stream cipher personalizado con un generador de números pseudoaleatorios para descifrar el texto cifrado, realizando un ataque de fuerza bruta para encontrar la semilla correcta que produce un texto que comienza con "FLAG_".

```
#!/usr/bin/env python3
```

```
import sys
import random
import binascii
```

```
def generate_keystream(seed, length):
```

```
    """Genera un flujo de clave usando un generador de números pseudoaleatorios
    random.seed(seed) # PRNG débil
    return bytes([random.randint(0, 255) for _ in range(length)])
```

```

def decrypt_with_seed(ciphertext, seed):
    """Intenta descifrar usando una semilla específica"""
    cipherbytes = bytes.fromhex(ciphertext)
    keystream = generate_keystream(seed, len(cipherbytes))
    plaintext = bytes([c ^ k for c, k in zip(cipherbytes, keystream)])
    return plaintext

def brute_force_decrypt(ciphertext, max_seed=100000, prefix=b"FLAG_"):
    """Fuerza bruta para encontrar la semilla correcta"""
    cipherbytes = bytes.fromhex(ciphertext)

    for seed in range(max_seed):
        keystream = generate_keystream(seed, len(cipherbytes))
        plaintext = bytes([c ^ k for c, k in zip(cipherbytes, keystream)])

        if plaintext.startswith(prefix):
            return seed, plaintext

    return None, None

def main():
    """Función principal del script"""
    if len(sys.argv) < 2 or len(sys.argv) > 3:
        print(f"Uso: {sys.argv[0]} <ruta_del_archivo> [semilla]")
        sys.exit(1)

    file_path = sys.argv[1]

    try:
        # Leer el contenido del archivo
        with open(file_path, 'r') as file:
            hex_content = file.read().strip()

        # Verificar que el contenido sea hexadecimal válido
        try:
            binascii.unhexlify(hex_content)
        except binascii.Error:
            print(f"Error: El contenido del archivo no es un hexadecimal válido")

```

```

        sys.exit(1)

# Verificar si se proporcionó una semilla
seed, plaintext = brute_force_decrypt(hex_content)

if seed is not None:
    print(f"Semilla encontrada: {seed}")
    try:
        decoded = plaintext.decode('utf-8')
        print(f"Texto descifrado: {decoded}")
    except UnicodeDecodeError:
        print(f"Texto descifrado (bytes): {plaintext}")
        print(f"Representación ASCII: {''.join(chr(b) if 32 <= b < 127 else '.' for b in plaintext)}")
    else:
        print("No se pudo encontrar una semilla válida (probando prefijo 'FLAG')")

# Intentar sin buscar un prefijo específico
for seed in range(10): # Probar algunas semillas bajas
    plaintext = decrypt_with_seed(hex_content, seed)
    try:
        decoded = plaintext.decode('utf-8')
        if all(32 <= ord(c) < 127 for c in decoded):
            print(f"Posible texto con semilla {seed}: {decoded}")
    except UnicodeDecodeError:
        pass

except FileNotFoundError:
    print(f"Error: El archivo {file_path} no existe")
    sys.exit(1)
except Exception as e:
    print(f"Error: {str(e)}")
    sys.exit(1)

if __name__ == "__main__":
    main()

```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).

```
usopp@c1ebee8fa1f1:~/ONEPIECE$ nano decrypt.py
usopp@c1ebee8fa1f1:~/ONEPIECE$ chmod +x decrypt.py
usopp@c1ebee8fa1f1:~/ONEPIECE$ ./decrypt.py ./Alabasta/Nanohana/Casa_de_Pell/flag.txt
Semilla encontrada: 1234
Texto descifrado: FLAG_241dc89f9cf87fd8c6fab03a76137d10
usopp@c1ebee8fa1f1:~/ONEPIECE$
```

Obteniendo así, la bandera descriptada.

```
FLAG_241dc89f9cf87fd8c6fab03a76137d10
```

Y la semilla utilizada

```
1234
```

Extraer párrafo de **poneglyph.jpeg**

Al igual que con la extracción de la bandera, el primer paso es encontrar la carpeta que contiene la imagen cifrada, para ello, utilizamos el cuarto comando documentado en  [Utils](#).

```
usopp@c1ebee8fa1f1:~/ONEPIECE$ find . -type f -name "*poneglyph*"
./Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom/poneglyph.zip
usopp@c1ebee8fa1f1:~/ONEPIECE$ cd ./Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom
usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom$ ls
poneglyph.zip
usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom$
```

Luego de obtener la carpeta, nos dirigimos hacia su directorio y la extraemos su contenido utilizando el quinto comando documentado en  [Utils](#). La contraseña a utilizar es la misma que la del usuario.

```

usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom$ 7z x poneglyph.zip -oponeglyph
7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=C,Utf16=off,HugeFiles=on,64 bits,8 CPUs Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz (806EA),ASM,AES-NI)

Scanning the drive for archives:
1 file, 64026 bytes (63 KiB)

Extracting archive: poneglyph.zip
--
Path = poneglyph.zip
Type = zip
Physical Size = 64026

Enter password (will not be echoed):
Everything is Ok

Size:          64208
Compressed:    64026
usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom$

```

Para poder utilizar el comando de extracción, es posible que sea necesario instalar ciertas librerías. Para ello, utilice el noveno comando documentado en  [Utils](#).

Para descifrar el parrafo, utilizamos un script que:

1. Extrae el texto de la imagen.
2. Toma el texto de la imagen y realiza una operación XOR entre el texto cifrado y mi carnet de estudiante (21542).

```

#!/bin/bash

#Verificar que se proporcione el carné como argumento
if [ $# -ne 1 ]; then
    echo "Uso: $0 <tu_carné>"
    exit 1
fi

STUDENT_ID=$1

#Definir los nombres de los directorios y archivos
IMAGE_PATH="poneglyph.jpeg" # Asumiendo que está en el mismo directorio
OUTPUT_FILE="decrypted.txt" # Archivo donde se guardará el texto descifrado

#Crear el archivo luffy_xor.py con el contenido correcto
cat > luffy_xor.py << 'EOF'

```

```

def xor_cipher(text, key):
    # Convertir tanto el texto como la clave a bytes
    if type(text) == str:
        text_bytes = text.encode()
    else:
        text_bytes = text
    key_bytes = key.encode() # Convertir la clave a bytes

    # Cifrar con XOR
    cipher_text = bytes([text_bytes[i] ^ key_bytes[i % len(key_bytes)] for i in range(len(text_bytes))])

    return cipher_text
EOF

#Crear un script Python temporal para la extracción
cat > extract_script.py << 'EOF'
from PIL import Image
import piexif
import sys
from luffy_xor import xor_cipher

def extraer_texto_metadata(imagen_path):
    # Abrir la imagen
    img = Image.open(imagen_path)

    # Obtener los metadatos EXIF
    try:
        exif_data = img.info.get('exif')
        if not exif_data:
            return None

        exif_dict = piexif.load(exif_data)

        # Obtener el texto almacenado en 'Artist'
        texto = exif_dict['0th'].get(piexif.ImageIFD.Artist)
        if texto:
            return texto
        return None

```

```

except Exception as e:
    print(f"Error al extraer metadatos: {e}")
    return None

# Obtener argumentos
if len(sys.argv) != 4:
    print("Uso: python script.py <imagen_path> <carne> <output_file>")
    sys.exit(1)

imagen_path = sys.argv[1]
student_id = sys.argv[2]
output_file = sys.argv[3]

# Extraer y descifrar
texto_cifrado = extraer_texto_metadata(imagen_path)
if texto_cifrado:
    try:
        # Aplicar XOR y decodificar si es posible
        texto_descifrado = xor_cipher(texto_cifrado, student_id)
        try:
            # Intentar decodificar como UTF-8
            decoded_text = texto_descifrado.decode('utf-8')

            # Guardar en archivo
            with open(output_file, 'w') as f:
                f.write(decoded_text)

            # Mostrar en pantalla
            print(decoded_text)

        except UnicodeDecodeError:
            # Si falla la decodificación UTF-8, guardar como bytes en archivo bina
            with open(output_file, 'wb') as f:
                f.write(texto_descifrado)

            # Mostrar en pantalla
            print(texto_descifrado)
            print(f"\nEl contenido binario descifrado se ha guardado en: {output_fil

```



```

except Exception as e:
    print(f"Error al descifrar: {e}")
else:
    print("No se encontraron metadatos EXIF o no hay campo Artist en la image
EOF

#Ejecutar el script en el mismo contenedor Docker
pip install pillow piexif

python3 extract_script.py "$IMAGE_PATH" "$STUDENT_ID" "$OUTPUT_FILE"

echo "Texto descifrado guardado en: $OUTPUT_FILE"

rm -f luffy_xor.py extract_script.py

```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).

```

usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom/poneglyph$ nano decryptImg.sh
usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom/poneglyph$ chmod +x decryptImg.sh
usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom/poneglyph$ ./decryptImg.sh "21542"
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pillow in /home/usopp/.local/lib/python3.10/site-packages (11.2.1)
Requirement already satisfied: piexif in /home/usopp/.local/lib/python3.10/site-packages (1.1.3)
however, he also admitted that saying what it was would take away the impact or any
Texto descifrado guardado en: decrypted.txt
usopp@c1ebee8fa1f1:~/ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Big_Mom/poneglyph$ █

```

Obteniendo así, el párrafo descriptado.

however, he also admitted that saying what it was would take away the impact